

ITERATION 3 – OBSERVER PATTERN

이벤트로 예약을 살아 움직이게

좌석 hold 만료, 결제 실패, 항공편 상태 변경 — 이전까지 명시적 호출로만
처리되던 부수효과를 Observer 패턴으로 분리합니다. Itinerary 환승 + 마
일리지 결제가 마지막 walking skeleton 칸을 메웁니다.

Observer

4 Subjects

4 Listeners

4 Event types

Air + Bus ticket

Connecting + MultiCity

Mileage payment

과목

ECE312 · 객체지향 설계패턴

팀

A — 김정욱 · 이재호 · 김경동

소스 베이스라인

KoreanAirReservationDomain

제출일

2026-05-14

walking skeleton

마지막 살을 채운다

iter1·2에서 채운 11개 기능 위에 iter3가 더하는 것 — Observer로 묶인 부수효과 4종, 6개 대도시 우등고속 버스티켓 연계, Itinerary, 마일리지 결제까지 6가지.

18

전체 기능 (CUMULATIVE)

6

ITER3 신규

4

OBSERVER 적용 자리

CATEGORY	SUB-FEATURE	ITER	패턴
Authentication	Member profile · 마일리지 잔액	ITER2	—
Authentication	Guest verification	ITER2	—
Booking Flow	Direct flight 검색·선택	ITER1	—
Booking Flow	Seat selection	ITER2	—
Reservation Lookup	Member 이력 / Guest 단건	ITER2	—
Cancellation	CancellationRequested → Cancelled	ITER2	STATE
Refund	환불 정책 선택 + 자동 환불	ITER2	STRATEGY
e-Ticket	e-Ticket 발권	ITER2	STATE
Hold Lifecycle	좌석 hold 15분 만료 자동 해제	ITER3 +	OBSERVER
Payment Failure	결제 실패 자동 취소 publisher	ITER3 +	OBSERVER
Flight Operations	FlightSchedule 변경 N개 전파	ITER3 +	OBSERVER
Air + Bus	6개 대도시 우등고속 버스티켓 연계 발매	ITER3 +	OBSERVER
Itinerary	Connecting + Multi-city · MCT 검증	ITER3 +	—
Payment	마일리지 결제 · MileageAccount 차감	ITER3 +	—

iter3가 더한 6개 확장기능

4개는 Observer 패턴 적용 — 부수효과를 호출자에서 떼어 listener로 분리. 특히 항공 e-Ticket 발급과 6개 대도시 우등고속 버스티켓 발매를 연계한다.

EXT-01 · OBSERVER

좌석 hold 만료 자동화

결제 미완료 좌석을 15분 후 자동 회수.
SeatHoldMonitor.sweep()이 만료 좌석을 발견하면
SeatHoldExpiredEvent를 발행하고,
ReservationHoldListener가 좌석을 해제하고 Reservation을
Cancelled로 전이.

SeatHoldMonitor → SeatHoldExpiredEvent →
ReservationHoldListener

EXT-02 · OBSERVER

결제 실패 자동 취소

iter2까지는 호출부가 직접 handlePaymentFailure를 호출했
다. iter3에서는 PaymentProcessor가 PaymentFailedEvent
를 발행하고 listener가 자동으로 전이를 수행.

PaymentProcessor → PaymentFailedEvent →
ReservationAutoCancelListener

EXT-03 · OBSERVER

FlightSchedule 변경 전파

관리자가 changeStatus(CANCELLED)를 호출하면 해당
schedule을 itinerary에 포함하는 모든 Reservation에 영향이
전파된다. AffectedReservationListener가
ReservationRegistry를 순회.

FlightSchedule → FlightStatusChangedEvent →
AffectedReservationListener

EXT-04 · OBSERVER

항공권 + 우등고속 버스티켓 연계

항공 e-Ticket 발급 완료 시 TicketIssuedEvent를 발행한다.
BusTicketPurchaseListener가 선택된 6개 대도시 중 하나의
우등고속 버스티켓을 자동 발매한다.

TicketPurchasePublisher → TicketIssuedEvent →
BusTicketPurchaseListener

EXT-05 · ITINERARY

Connecting + Multi-city

Itinerary.connecting(first, second)로 2-leg 환승 일정을 만
든다. isConnectionTimeValid(MCT)가 환승 시간이 최소 60
분(국내) / 90분(국제) 이상인지 검증.

Itinerary.DIRECT | CONNECTING | MULTI_CITY · MCT
60 / 90

EXT-06 · PAYMENT

마일리지 적용 결제

PaymentMethod.MILEAGE 신규 enum.
PaymentProcessor.processMileagePayment(account,
cost, pnr)가 MileageAccount.withdraw로 차감하고 실패 시
PaymentFailedEvent 발행 — 일반 결제 실패 경로와 동일하게
자동 취소된다. MockSkypassInterface가 외부 Skypass 시
스템의 in-memory mock을 제공.

PaymentMethod.MILEAGE · MileageAccount.withdraw ·
MockSkypassInterface

패턴이 도입되는 자리, 도입 이유

각 iteration에서 채워야 할 요구사항(R)과 그 요구사항을 깔끔하게 처리하기 위해 도입한 디자인 패턴(DP)의 매핑. iter3는 항공권 구매 후 버스티켓 연계 발매까지 포함해, 이벤트 기반 부수효과가 늘어나는 시점이라 Observer가 자연스럽게 들어온다.

ITER	REQUIREMENT (R)	DESIGN PATTERN (DP)	진입점 클래스	도입 동기
ITER1	Booking 생애주기 전이	State	Reservation.currentState	전이 분기 폭발 방지
ITER1	Walking skeleton happy path	—	BookingController	한 갈래 path 우선
ITER2	운임별 환불 정책	Strategy	RefundPolicy family	운임 추가 = 정책 추가 (OCP)
ITER2	Cancel/Refund 5개 전이	State 확장	CancelledState · RefundRequestedState	iter1 State 확장
ITER3 +	좌석 hold 만료 자동 처리	Observer	SeatHoldMonitor · ReservationHoldListener	호출자 vs 부수효과 분리
ITER3 +	결제 실패 → 자동 취소	Observer	PaymentProcessor · ReservationAutoCancelListener	실패 경로 일관 처리
ITER3 +	FlightSchedule 변경 전파	Observer	FlightSchedule · AffectedReservationListener	1-to-N 통지
ITER3 +	6개 대도시 우등고속 버스티켓 연계 발매	Observer	TicketPurchasePublisher · BusTicketPurchaseListener	항공권 구매 완료 이벤트 연계
ITER3 +	Connecting · Multi-city	— (도메인 확장)	Itinerary · ItinerarySearchService	Segment 다중화 + MCT
ITER3 +	마일리지 결제	— (도메인 확장)	MileageAccount · MockSkypassInterface	결제 수단 분기

누가, 어떤 책임을 맡았나

iter3 핵심 6종을 3인 분담. Observer는 publisher와 listener 양쪽을 함께 설명하고, 버스티켓 연계는 항공권 발권 이벤트의 추가 listener로 묶었다.



김정욱

OBSERVER INFRA · HOLD MONITOR

Observer 인프라 + 좌석 hold 만료 자동화

- domain/event 패키지 (DomainEvent · EventPublisher · EventListener)
- SeatHoldMonitor.sweep + Seat.holdExpiresAt
- ReservationHoldListener — release + handlePaymentFailure
- deck Vercel-light 디자인 토큰



이재호

PAYMENT + FLIGHT PROPAGATION

결제 실패 + FlightSchedule 변경 publisher

- PaymentProcessor extends EventPublisher
- PaymentFailedEvent + ReservationAutoCancelListener
- FlightSchedule.changeStatus publish
- AffectedReservationListener + ReservationRegistry

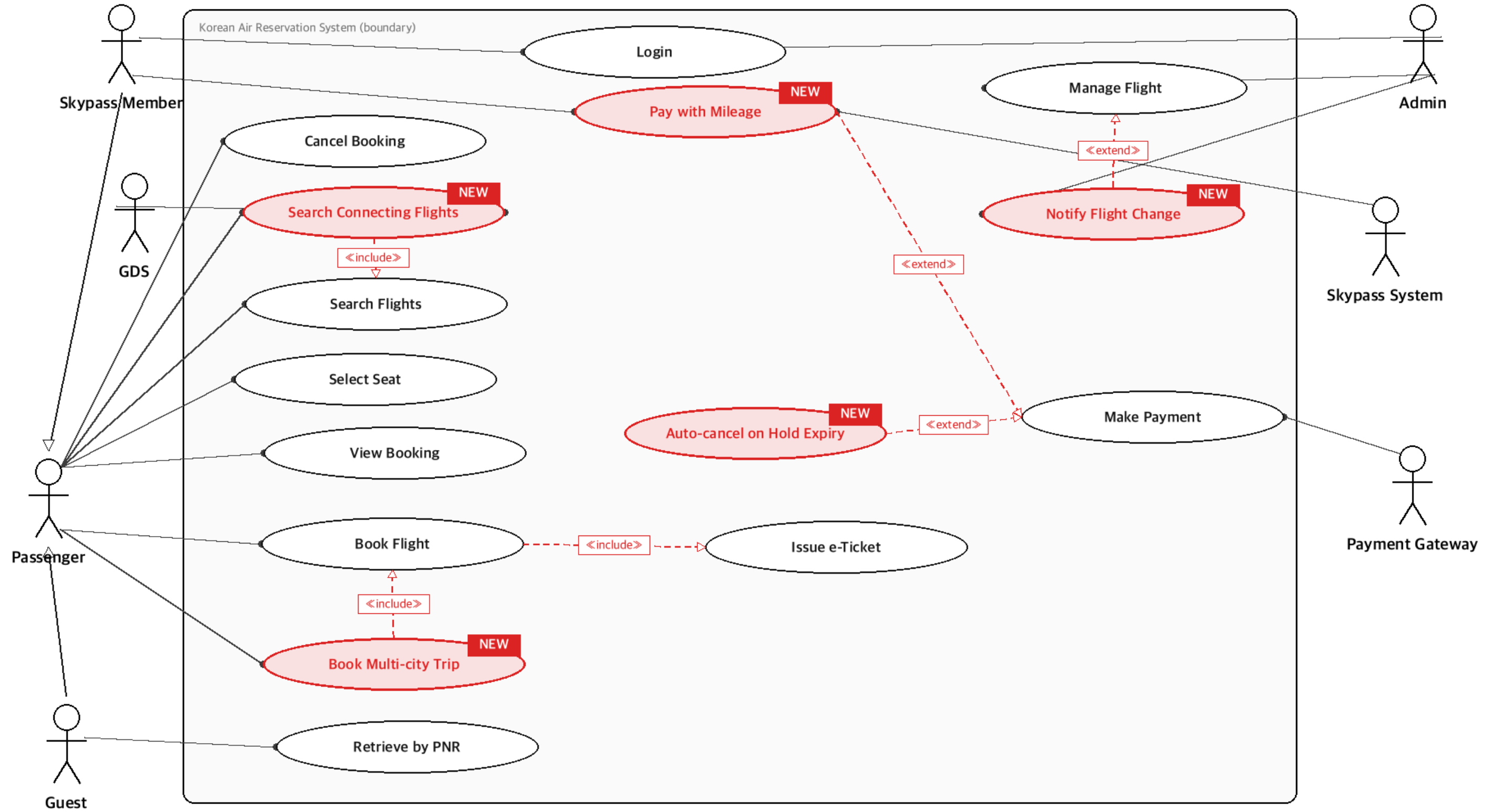


김경동

TICKET ADD-ON · ITINERARY · MILEAGE

항공권 연계 버스티켓 + Connecting flight + 마일리지 결제

- TicketPurchasePublisher + BusTicketPurchaseListener
- Itinerary.connecting / multiCity + MCT 검증
- ItinerarySearchService — 1-stop 조합 탐색
- PaymentMethod.MILEAGE + processMileagePayment



NEW iter3 신규 use case (5)

○ iter1/2 기존 use case

— actor — UC association

△ actor generalization →▷

<<include>> / <<extend>> (dashed → UC)

발표 시연용 6개 흐름

각 시나리오는 발표 시 콘솔/Swing에서 끝까지 동작하는 단위. 4개 Observer + Connecting + Mileage. 버스티켓 연계는 항공권 구매 완료 이벤트에 listener가 반응하는 핵심 시연이다.

SC-01 · OBSERVER

좌석 hold 만료 자동 해제

TRIGGER · SCHEDULER.SWEEP()

01

회원이 좌석 선택 → `Seat.hold(15, pnr)` 로 `holdExpiresAt` 세팅

02

결제 단계로 이동하지 않은 채 15분 경과

03

`SeatHoldMonitor.sweep()` 이 만료 좌석 감지

04

`SeatHoldExpiredEvent` 발행 → listener가 `release + handlePaymentFailure`

05

좌석 Available, 예약 Cancelled로 종결

SC-02 · OBSERVER

결제 실패 자동 취소

TRIGGER · GATEWAY.AUTHORIZE() → FALSE

01

회원이 결제 정보 입력 후 `submitPayment` 호출

02

`processPaymentCharge(amount, pnr)` 실행

03

`gateway.authorize()` 거절 → `payment.fail()`

04

`PaymentFailedEvent("gateway-declined")` 발행

05

`AutoCancelListener`가 `PendingPayment` → `Cancelled`

SC-03 · OBSERVER

FlightSchedule 변경 전파

TRIGGER · ADMIN.CHANGEFLIGHTSTATUS(KE001, CANCELLED)

01

관리자가 항공편 취소 명령 입력

02

`FlightSchedule.changeStatus(CANCELLED)` — 내부 publish

03

`FlightStatusChangedEvent` 가 prev/new status 담아 broadcast

04

`AffectedReservationListener`가 registry에서 `schedule` 참조 예약 필터링

05

각 Reservation에 `evaluateImpact` 호출 + 콘솔 로그

SC-04 · OBSERVER

항공권 구매 연계 우등고속 버스티켓 발매

TRIGGER · E-TICKET ISSUED + BUS CITY SELECTED

01

회원이 항공권 결제 후 `Reservation.issueTicket()` 으로 e-Ticket 발급

02

사용자가 6개 대도시 중 버스 목적지 선택: 서울·부산·대구·광주·대전·인천

03

`TicketPurchasePublisher.publishTicketIssued(..., BusCity)` 호출

04

`BusTicketPurchaseListener.onEvent` 가 이벤트 수신

05

`BusTicketingService.issuePremiumTicket` 으로 연계 버스티켓 발매

SC-05 · ITINERARY

환승 일정 MCT 검증

TRIGGER · 사용자: ICN → JFK (DATE)

01

UI가 `searchConnecting(from, to, date, MCT)` 호출

SC-06 · MILEAGE PAYMENT

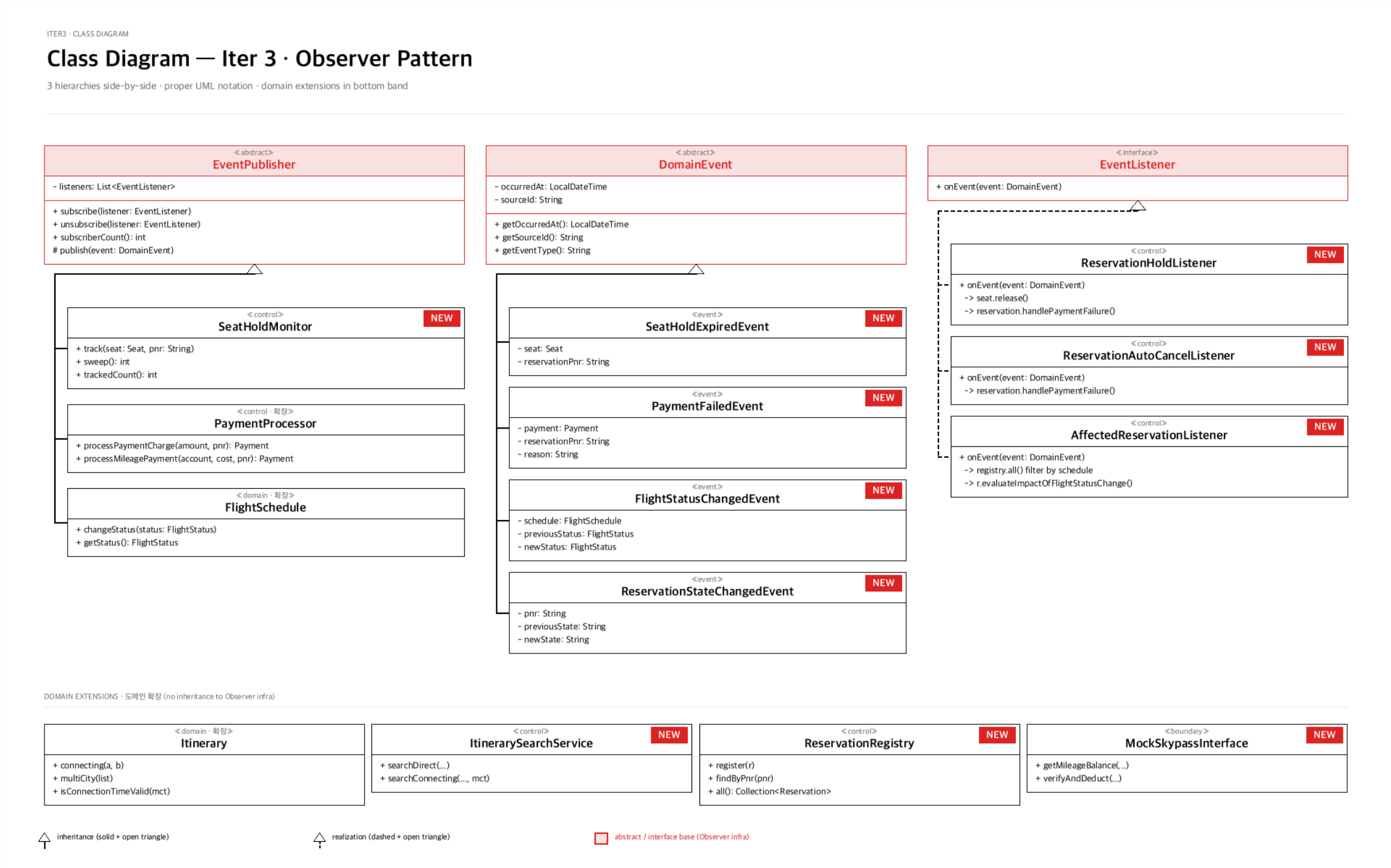
마일리지 결제 + 잔액 부족 시 자동 취소

TRIGGER · 회원: PAYWITHMILEAGE(RESERVATION, MILEAGECOST)

01

회원이 결제 화면에서 "마일리지 결제" 선택, `cost` 입력

Observer 인프라 + Bus add-on까지 — NEW badge로 표시



NEW

iter3 신규
(12개)

□

Observer abstract /
interface base

▲

solid line + 삼각형 =
inheritance

dashed line + 삼각형 =
realization

side spine routing — 화살표가 concrete
box 위 통과 안 함

발표자가 가리킬 진입점 메서드

Subject 4종과 Listener 4종 + Itinerary/Mileage 확장. 항공권 구매 연계 버스티켓은 TicketIssuedEvent를 기준으로 설명한다.

클래스	ECB 역할	ITER3 핵심 메서드
DomainEvent (abstract)	DOMAIN · EVENT	occurredAt , sourceId , getEventType() — 모든 이벤트 베이스
EventListener (interface)	DOMAIN · OBSERVER	onEvent(DomainEvent event) — 구현체가 instanceof 분기
EventPublisher (abstract)	DOMAIN · SUBJECT	subscribe(listener) , unsubscribe(listener) , publish(event)
SeatHoldMonitor	CONTROL · SUBJECT	track(seat, pnr) , sweep() — 만료 좌석 검출 후 SeatHoldExpiredEvent 발행
ReservationHoldListener	CONTROL · LISTENER	onEvent(event) — Seat.release + Reservation.handlePaymentFailure
PaymentProcessor (확장)	CONTROL · SUBJECT	extends EventPublisher · processPaymentCharge(amount, pnr) · processMileagePayment(account, cost, pnr)
ReservationAutoCancelListener	CONTROL · LISTENER	onEvent(event) — Reservation.findByPnr + handlePaymentFailure
FlightSchedule (확장)	DOMAIN · SUBJECT	extends EventPublisher · changeStatus(newStatus) 내부에서 FlightStatusChangedEvent 발행
AffectedReservationListener	CONTROL · LISTENER	onEvent(event) — registry.all() 순회 → schedule 참조 Reservation 필터링
TicketPurchasePublisher	CONTROL · SUBJECT	publishTicketIssued(reservation, ticket, busCity) — 항공권 발급 완료 이벤트 발행
BusTicketPurchaseListener	CONTROL · LISTENER	onEvent(event) — TicketIssuedEvent 수신 → BusTicketingService 호출
BusTicketingService	CONTROL · SERVICE	supportedCities() , issuePremiumTicket(...) — 6개 대도시 우등고속 발매
ReservationRegistry	CONTROL · REGISTRY	register , findByPnr , all() — 활성 Reservation 전수 순회
Itinerary (확장)	DOMAIN · ENTITY	connecting(first, second) , multiCity(list) , isConnectionTimeValid(MCT)
ItinerarySearchService	CONTROL · SERVICE	searchDirect(from, to, date) , searchConnecting(from, to, date, MCT)
Seat (확장)	DOMAIN · ENTITY	hold(timeoutMinutes, pnr) · holdExpiresAt · isHoldExpired()
MockSkypassInterface	BOUNDARY · MOCK	verifyMembership , getMileageBalance , deductMileage , verifyAndDeduct

교과서 정의 옆에 우리 매핑을 둔다

Observer Pattern

한 객체의 상태가 변할 때 그것에 의존하는 모든 객체에게 통지되어 자동으로 갱신되도록 하는 1-to-N 의존성을 정의한다.

SUBJECT 관찰 대상. attach, detach, notify API

OBSERVER 통지 인터페이스. `update(subject)` 추상 메서드

CONCRETESUBJECT실제 상태 변경 클래스. 변경 시 notify 호출

Observer 구현체. update에서 subject 상태 가져와 갱신

왜 쓰나 — Subject와 Observer를 느슨하게 결합. Subject가 Observer 구현 클래스를 몰라도 통지 가능. 새 Observer 추가가 Subject 코드 수정 없이 가능 (OCP).

47|| Subject · 47|| Listener

좌석 hold 만료 · 결제 실패 · 항공편 변경 · 항공권 구매 연계 버스티켓 — 모두 "발행자는 부수효과를 모르고, listener가 자신의 책임만 수행"하는 구조로 통일.

SUBJECT (BASE) `EventPublisher` abstract — subscribe / unsubscribe / publish

OBSERVER (BASE) `EventListener` interface — `onEvent(DomainEvent)`

CONCRETESUBJECT: SeatHoldMonitor · PaymentProcessor · FlightSchedule
· TicketPurchasePublisher

```
CONCRETEOBSER' ReservationHoldListener · AutoCancelListener ·  
AffectedReservationListener ·  
BusTicketPurchaseListener
```

EVENT
PAYLOAD

push-model — DomainEvent 서브클래스가 데이터 담아 전달

왜 도입했나 — iter2까지는 호출자(BookingController)가 부수효과를 매번 직접 호출했다. iter3에서 항공권 구매 후 버스티켓 발매처 럼 연계 부수효과가 추가되면서 호출자가 비대해질 수 있어, Observer로 부수효과를 listener에 위임해 단일 책임으로 분리.

호출자가 부수효과를 직접 호출 → listener 위임

결제 실패 후 Reservation을 Cancelled로 전이하는 책임이 iter2까지는 BookingController에 있었다. iter3에서는 publisher / listener가 그 부담을 나눠 가진다.

BEFORE · ITER2 · 호출자 책임

BookingController.confirmPayment

```
public Payment confirmPayment(Reservation r,
                               FareRule rule,
                               long baseFare,
                               long tax) {
    if (!paymentProcessor.validateFareRule(rule)) {
        throw new IllegalArgumentException(...);
    }
    long total = paymentProcessor.calculateTotalAmount(baseFare, tax);
    Payment payment = paymentProcessor.processPaymentCharge(total);

    if (payment.getStatus() == PaymentStatus.PAID) {
        r.addPayment(payment);
        r.processPayment();
    } else {
        // 호출자가 직접 전이를 책임진다
        r.handlePaymentFailure();
    }
    return payment;
}
```

문제 — 동일한 "실패 시 Cancelled 전이"가 마일리지 결제, 좌석 hold 만료, 항공편 취소에서도 필요. 매번 호출자에서 같은 if/else를 반복해야 한다.

AFTER · ITER3 · PUBLISH + LISTENER

PaymentProcessor + Listener

```
// PaymentProcessor - Subject
public class PaymentProcessor extends EventPublisher {
    public Payment processPaymentCharge(long amount, String pnr) {
        ...
        if (authorized) {
            payment.pay();
        } else {
            payment.fail();
            publish(new PaymentFailedEvent(
                payment, pnr, "gateway-declined"));
        }
        return payment;
    }
}

// ReservationAutoCancelListener - Observer
public class ReservationAutoCancelListener
    implements EventListener {
    public void onEvent(DomainEvent e) {
        if (!(e instanceof PaymentFailedEvent ev)) return;
        Reservation r = Reservation.findByPnr(ev.getReservationPnr());
        if (r != null) r.handlePaymentFailure();
    }
}
```

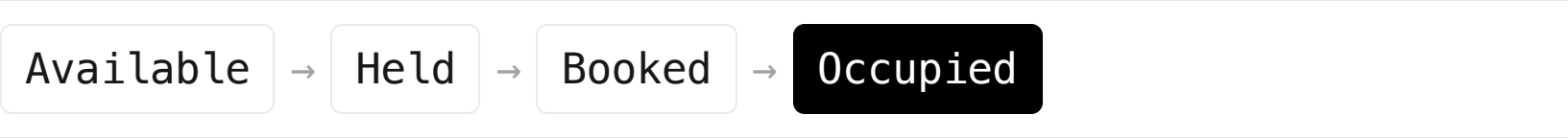
변화 — BookingController에서 `handlePaymentFailure()` 직접 호출 사라짐. 새 발생원(마일리지 실패, 항공권 구매 후 버스티켓 발매)도 publisher가 이벤트만 발행하면 listener가 동일 구조로 반응.

상태는 그대로, 트리거 방식이 바뀐다

iter3는 Reservation 상태를 추가하지 않는다. 대신 Seat과 FlightSchedule의 전이가 이벤트 발행 부수효과를 동반한다. 상태 다이어그램은 동일하지만 트리거 라벨이 달라진다.

SEAT LIFECYCLE

Seat — hold 만료 자동 해제



iter3 신규 전이

Held → Available · **SeatHoldMonitor.sweep()**
→ **SeatHoldExpiredEvent** publish
→ **ReservationHoldListener.onEvent** → seat.release()

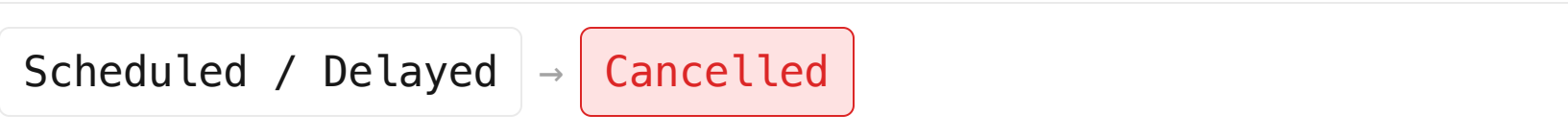
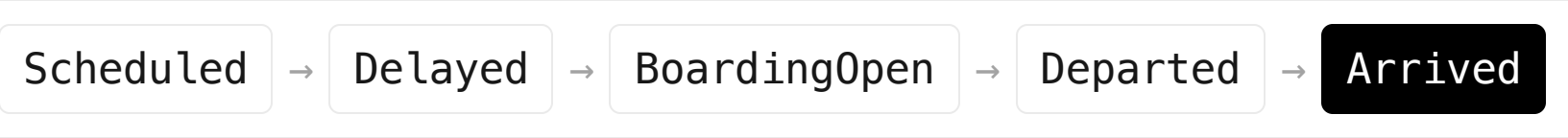
Seat 신규 필드

holdExpiresAt : LocalDateTime
heldByPnr : String
isHoldExpired() : boolean

핵심 — 만료 판정은 Seat 본인이 하고, 부수효과(예약 취소)는 listener가 가져간다. Seat이 Reservation을 직접 알지 못한다.

FLIGHT SCHEDULE LIFECYCLE

FlightSchedule — 전파 책임 분리



iter3 전이 부수효과

changeStatus(newStatus) — 모든 전이에서
→ **FlightStatusChangedEvent(prev, new)** publish
→ **AffectedReservationListener** 가 registry.all() 순회

왜 1:N 전파인가

한 schedule을 itinerary에 포함하는 Reservation이 N개. DELAYED · CANCELLED · DEPARTED 어떤 상태든 모든 N에게 통지가 필요. Observer가 정확히 1-to-N notification 시나리오.

핵심 — FlightSchedule이 ReservationRegistry를 직접 알지 않는다. publish만 하고, listener가 registry를 owns한다.

6개 흐름 — Observer 4개가 중심

SC-04 버스티켓 연계는 별도 다이어그램 대신 Demo + Pattern Code Map에서 집중 설명한다. 핵심은 TicketIssuedEvent에 BusTicketPurchaseListener가 반응하는 Observer 구조다.

SC-01 · OBSERVER

Seat Hold Expiry

SeatHoldMonitor.sweep() →
SeatHoldExpiredEvent →
ReservationHoldListener →
seat.release() + reservation
cancel.

SLIDE 14

SC-02 · OBSERVER

Payment Failure

processPaymentCharge →
gateway 거절 →
PaymentFailedEvent →
AutoCancelListener →
handlePaymentFailure.

SLIDE 15

SC-03 · OBSERVER

Flight Propagation

changeStatus(CANCELLED) →
FlightStatusChangedEvent →
AffectedReservationListener →
registry.all() filter → 통지.

SLIDE 16

SC-04 · OBSERVER

Air + Bus Ticket

e-Ticket issued →
TicketIssuedEvent →
BusTicketPurchaseListener →
issuePremiumTicket(6 cities).

SLIDE 19 / 23

SC-05 · ITINERARY

Connecting Search

searchConnecting → catalog 순
회 → Itinerary.connecting(a, b) →
isConnectionTimeValid(MCT) →
통과만 반환.

SLIDE 17

SC-06 · MILEAGE

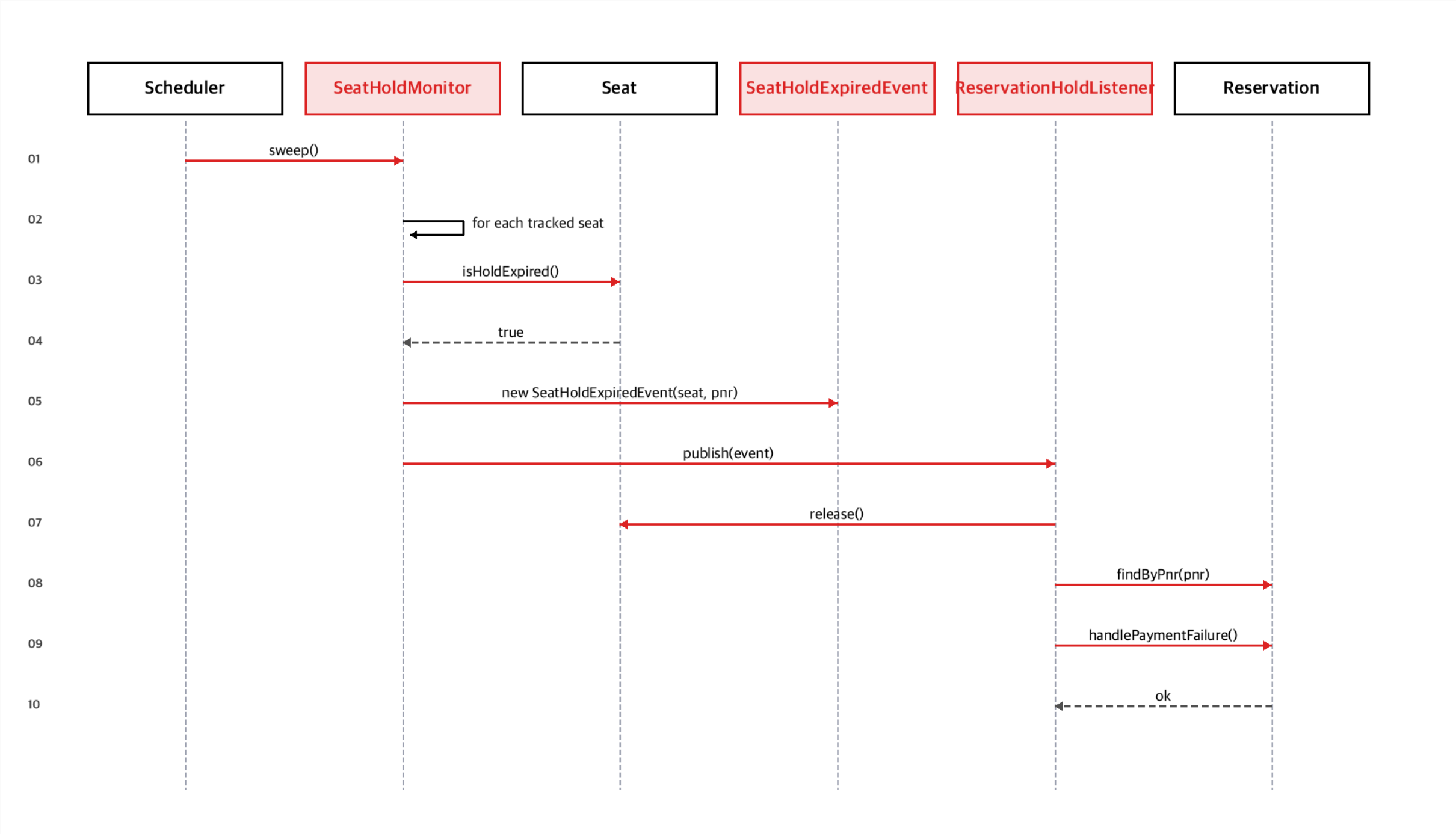
Mileage Payment

processMileagePayment →
MileageAccount.withdraw →
SkypassInterface.verifyAndDeduct
→ addPayment +
processPayment.

SLIDE 18

SHARED PATTERN

4개 Observer 흐름 (SC-01·02·03·04) 모두 동일한 모양:
Subject가 publish → EventPublisher가 broadcast → Listener.onEvent가 instanceof 분기 → 도메인 메서드 호출.



TRIGGER

scheduler.sweep()

실서비스: ScheduledExecutorService 주기 호출 · 학습 프로젝트: 데모 코드가 직접 호출.

CRITICAL POINT

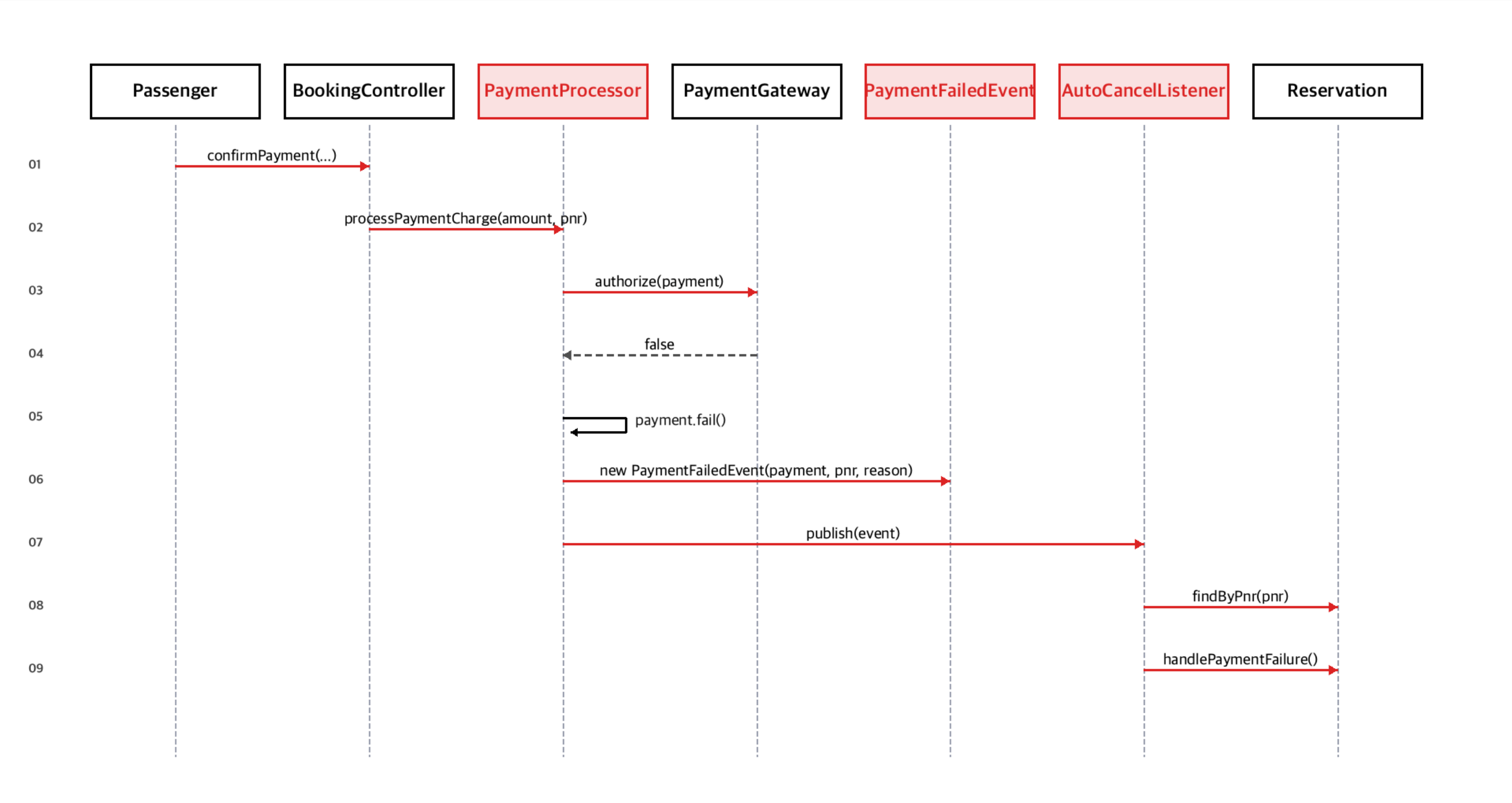
SeatHoldMonitor.sweep()

tracked seat 순회 → `seat.isHoldExpired()` true → publish. SeatHoldMonitor가 Reservation을 직접 모름.

LISTENER 책임

ReservationHoldListener

`seat.release()` + `reservation.handlePaymentFailure()` . 두 부수효과를 한 곳에 모음.



TRIGGER

gateway.authorize() = false

학습 프로젝트: FailingGateway.declineNext = true · 실 시나리오: 카드 한도 초과·네트워크 타임아웃.

VS ITER2

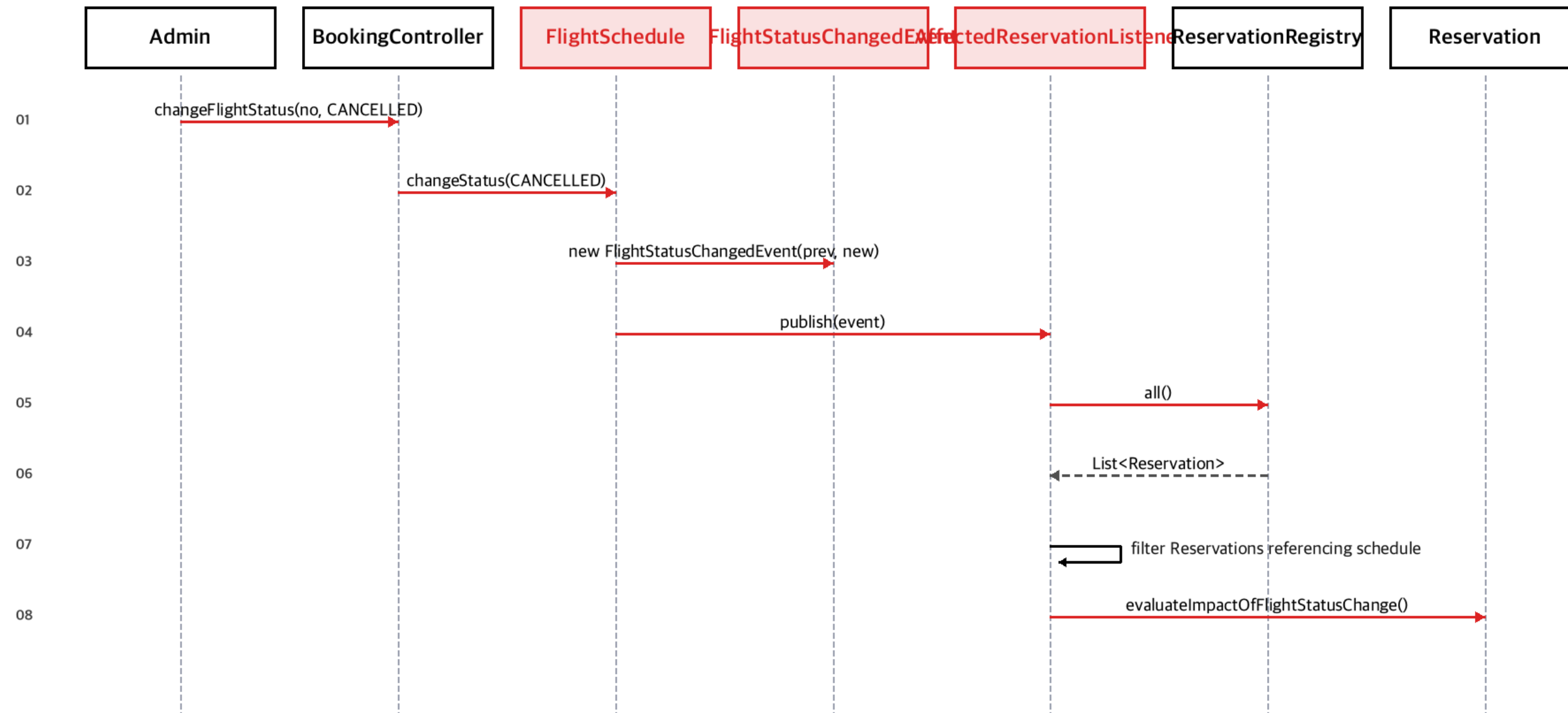
호출자 직접 호출 사라짐

iter2 BookingController에 if/else 분기 있었음 → iter3: PaymentProcessor가 publish만. 마일리지(SC-06)도 같은 listener 재사용.

EVENT PAYLOAD

PaymentFailedEvent

payment · reservationPnr · reason (gateway-declined / insufficient-mileage)



TRIGGER

admin.changeFlightStatus(KE001, CANCELLED)

관리자 콘솔 명령 → FlightSchedule.changeStatus 진입점 → 내부에서 publish.

왜 1-T0-N 인가

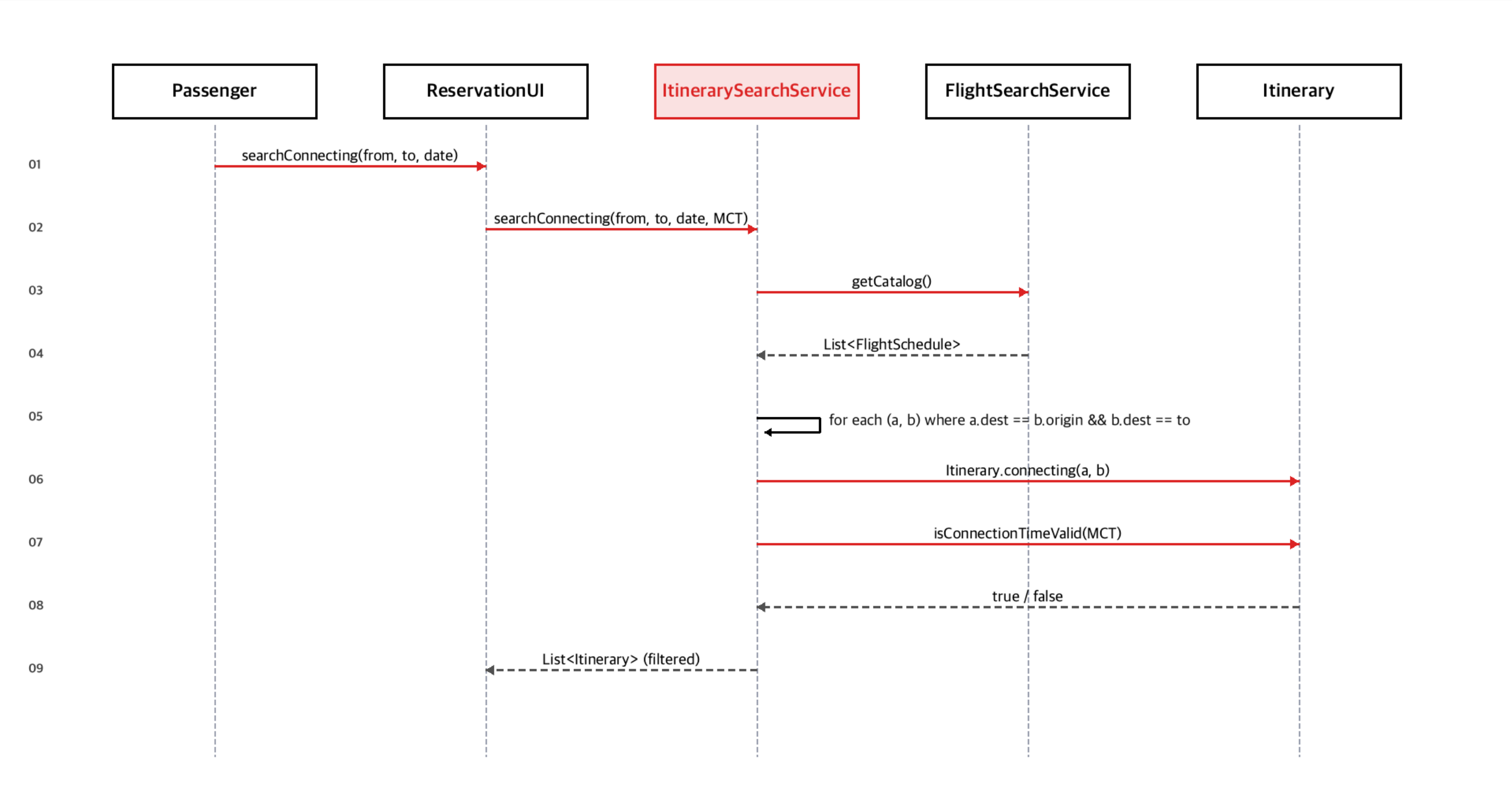
한 schedule을 N개 Reservation 참조

일정 변경이 모든 N에 전파 필요. Observer 가장 자연스러운 시나리오. FlightSchedule이 ReservationRegistry를 직접 모름.

LISTENER 책임

AffectedReservationListener

registry.all() → schedule 참조 Reservation 필터 → evaluateImpactOfFlightStatusChange() 호출.



TRIGGER

user: searchConnecting(ICN, JFK, date)

UI → ItinerarySearchService. MCT 기본 90분 (국제) / 60분 (국내).

알고리즘

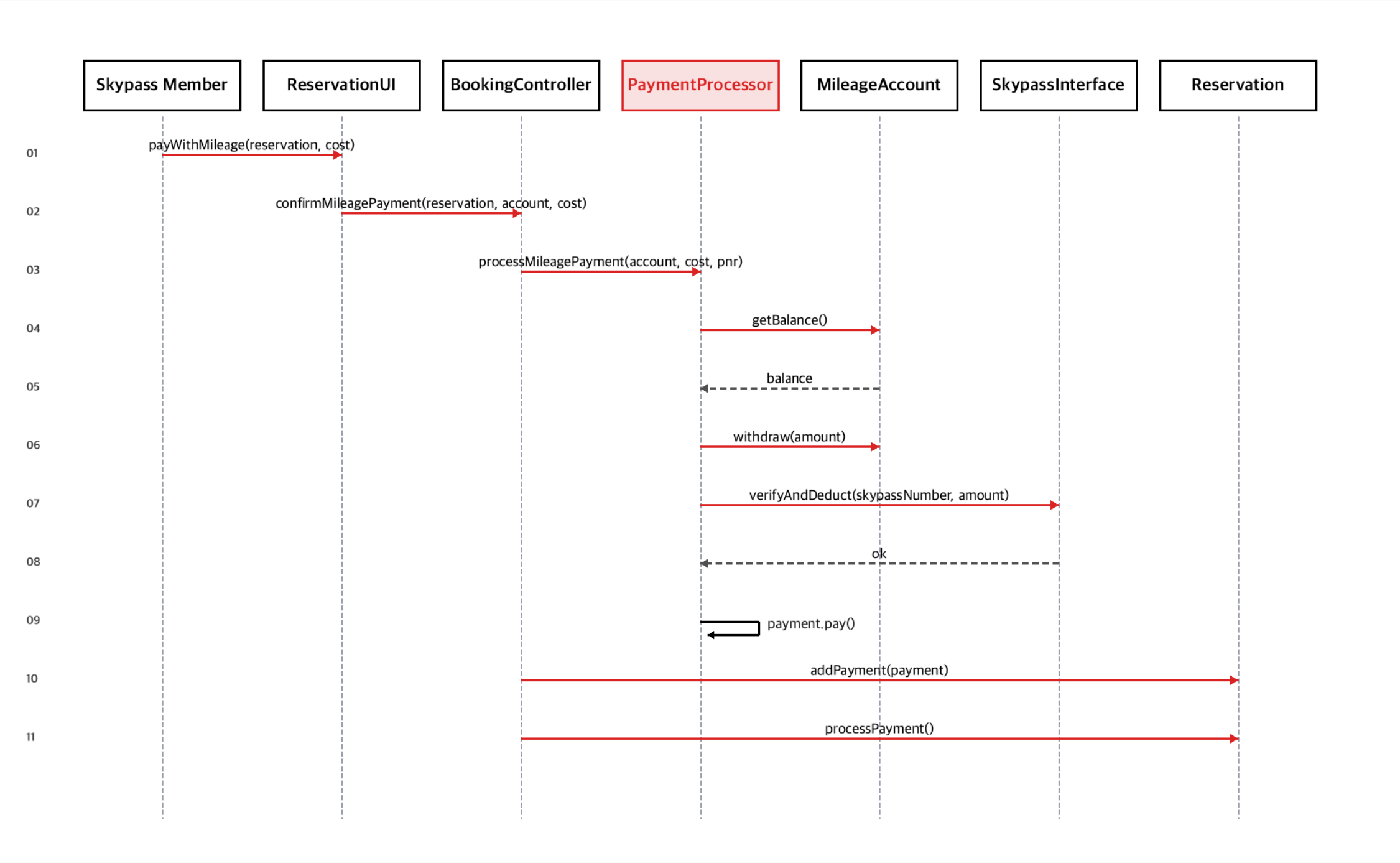
1-stop pair 탐색

catalog 순회 → a.dest == b.origin && b.dest == JFK 인 (a, b) → Itinerary.connecting(a, b)
→ MCT 통과만.

CRITICAL CHECK

isConnectionTimeValid

layover = arrival(a) ↔ departure(b). layover ≥ MCT → accept. layover < MCT → reject.



TRIGGER

member: payWithMileage(reservation, cost)

UI → BookingController.confirmMileagePayment. PaymentMethod.MILEAGE 신규 enum.

FLOW

2-step debit

(1) MileageAccount.withdraw 로컬 차감 → (2) SkypassInterface.verifyAndDeduct 외부 검증 → 둘 다 성공 시 Confirmed 전이.

FAILURE PATH (재사용)

잔액 부족 → SC-02와 동일

PaymentFailedEvent("insufficient-mileage") → AutoCancelListener가 동일 listener로 처리 → Observer 인프라 재사용 = 도입 가치 입증.

실행 명령: `java -cp bin com.koreanair.reservation.app.Iter3DemoRunner` — 아래는 실제 콘솔 출력 그대로. mockup 없이 진짜 stdout 캡처.

```
[BOOT] event subscribers loaded
SeatHoldMonitor .subscriberCount = 1
PaymentProcessor .subscriberCount = 1
TicketPublisher .subscriberCount = 1
→ ReservationHoldListener registered
→ ReservationAutoCancelListener registered
→ BusTicketPurchaseListener registered
→ AffectedReservationListener attached per FlightSchedule
```

SC-01 · OBSERVER

Seat hold expiry

```
[STATE] Initiated -> PendingPayment
[INIT] tracked seat 12A
(PNR=PNR-DEM001,
holdExpiresAt=...10:19:36)
[HOLD-EXPIRY] seat 12A released
[STATE] PendingPayment -> Cancelled
[HOLD-EXPIRY] PNR=PNR-DEM001
auto-cancelled
[SWEEP] fired=1 events
[RESULT] seat=12A state=AVAILABLE
reservation=Cancelled
```

SC-02 · OBSERVER

Payment failure

```
[STATE] Initiated -> PendingPayment
[GATEWAY] authorize -> false
(amount=320000)
[STATE] PendingPayment -> Cancelled
[AUTO-CANCEL] PNR=PNR-DEM002
reason=gateway-declined
[RESULT] payment.status=FAILED
reservation=Cancelled
```

SC-03 · OBSERVER

Flight propagation

```
[ADMIN] changeFlightStatus(KE001,
CANCELLED)
[FLIGHT-CANCEL] PNR=PNR-DEM003A
flight=KE001
SCHEDULED -> CANCELLED
[FLIGHT-CANCEL] PNR=PNR-DEM003B
flight=KE001
SCHEDULED -> CANCELLED
[RESULT] schedule.status=CANCELLED
```

SC-04 · OBSERVER

Air + premium bus ticket

```
[STATE] Initiated -> PendingPayment
[STATE] PendingPayment -> Confirmed
[TICKET] e-Ticket KE-260515-0001
issued for Kim Jaeho
[STATE] Confirmed -> Ticketed
[BUS] premium bus ticket
KOBUS-260515-0001 issued
city=부산 pnr=PNR-DEM004-BUS
[RESULT] linkedBusTickets=1
reservation=Ticketed
```

동일한 `Iter3DemoRunner` 가 마일리지 결제와 connecting itinerary MCT 검증까지 검증. 출력 원본은 `assets/iter3-demo-output.txt`.

SC-06 · MILEAGE PAYMENT

PaymentProcessor.processMileagePayment

```
[STATE] Initiated -> PendingPayment [MILEAGE] balance before =
50000 [MILEAGE] balance after = 20000 [MILEAGE] external
skypass.deduct = true [STATE] PendingPayment -> Confirmed
[RESULT] payment=PAID reservation=Confirmed
```

검증 — MileageAccount.withdraw(30,000)으로 잔액 50k → 20k 차감. 외부 SkypassInterface.verifyAndDeduct가 true 반환 → Confirmed 전이. 잔액 부족 시 PaymentFailedEvent → SC-02 listener가 처리 (재사용).

SC-05 · ITINERARY MCT

Itinerary.isConnectionTimeValid

```
[ICN→NRT→JFK 120m layover] valid=true [ICN→NRT→JFK 20m layover
] valid=false
```

검증 — MCT(국제 90분) 검증. 120분 layover는 통과, 20분 layover는 거절. ItinerarySearchService.searchConnecting가 통과한 후보만 결과에 포함.

State는 "상태별 허용 메서드"를 바로 보여준다

교과서의 Context / State / ConcreteState가 코드에서 그대로 드러난다. 발표 때는 왼쪽 매핑을 먼저 짚고, 오른쪽 코드에서 `currentState.method(this)` 와 `ctx.setState(...)` 만 보여주면 된다.

GOF ROLE → TEAM CLASS

Reservation lifecycle

Context	Reservation 현재 상태를 <code>currentState</code> 로 보관
State	ReservationState 8개 lifecycle 메서드 선언
Concrete	ConfirmedState , TicketedState , CancelledState 등
Transition	구체 상태가 <code>ctx.setState(new NextState())</code> 호출
Confirmed <code>issueTicket()</code> Ticketed <code>requestCancellation()</code>	
CancellationRequested <code>confirmCancellation()</code> Cancelled	

ACTUAL CODE SLICE

메서드는 Context에 있지만 판단은 State가 한다

```
// Reservation.java - Context
private ReservationState currentState;

public Reservation() {
    this.currentState = new InitiatedState();
}

public void issueTicket() {
    currentState.issueTicket(this);
}

public void requestCancellation() {
    currentState.requestCancellation(this);
}

public void setState(ReservationState next) {
    System.out.printf("[STATE] %s -> %s\n",
        currentState.name(), next.name());
    currentState = next;
}

// ConfirmedState.java - ConcreteState
public void issueTicket(Reservation ctx) {
    // Ticket.generate(...) 후
    ctx.setState(new TicketedState());
    ctx.updateStatus(ReservationStatus.TICKETED);
}
```

Strategy는 “환불 계산식”을 정책 클래스로 분리한다

RefundHandler 는 어떤 정책인지 고른 뒤, 계산은 RefundPolicy 인터페이스에 맡긴다. 그래서 발표에서는 paid 가 결제 총액이고, concrete policy마다 반환식이 달라진다고 설명하면 된다.

GOF ROLE → TEAM CLASS

RefundPolicy family

Context	RefundHandler — 환불 흐름 orchestration
Strategy	RefundPolicy — calculateRefundAmount(paid)
Concrete	FullRefundPolicy , PartialRefundPolicy , NoRefundPolicy
Selection	resolvePolicy(fareRule) — fare class 기준 매핑

Full	Partial	No
입력 paid	입력 paid	입력 paid
출력 paid	출력 paid × 0.5	출력 0
전액 환불	부분 환불	환불 불가

ACTUAL CODE SLICE

Context는 인터페이스만 호출한다

```
// RefundHandler.java - Context
RefundPolicy policy = resolvePolicy(fareRule);
BigDecimal paid = BigDecimal.ZERO;
for (Payment p : reservation.getPayments()) {
    if (p != null && p.getAmount() != null) {
        paid = paid.add(p.getAmount());
    }
}

BigDecimal refundAmount =
    policy.calculateRefundAmount(paid);

private RefundPolicy resolvePolicy(FareRule rule) {
    if (rule == null || !rule.isRefundable()) {
        return new NoRefundPolicy();
    }
    if ("Y".equals(rule.getFareClass())
        || "B".equals(rule.getFareClass())) {
        return new FullRefundPolicy();
    }
    return new PartialRefundPolicy();
}

// RefundPolicy.java - Strategy
interface RefundPolicy {
    BigDecimal calculateRefundAmount(BigDecimal paid);
}
```

Observer는 “발행자는 publish만, listener가 반응”한다

iter3의 핵심은 호출자가 부수효과를 직접 처리하지 않는 것이다. Subject는 이벤트만 발행하고, 구독자는 관심 이벤트만 골라 상태 전이/좌석 해제/영향 예약 조회/연계 버스티켓 발매를 수행한다.

GOF ROLE → TEAM CLASS

EventPublisher + Listener

Subject	EventPublisher subscribe, unsubscribe, publish
Observer	EventListener onEvent(DomainEvent)
ConcreteSubject	SeatHoldMonitor, PaymentProcessor, FlightSchedule, TicketPurchasePublisher
ConcreteObserver	ReservationHoldListener, ReservationAutoCancelListener, AffectedReservationListener, BusTicketPurchaseListener

SeatHoldMonitor.sweep() → publish(SeatHoldExpiredEvent) → ReservationHoldListener.onEvent → seat.release() + r.handlePaymentFailure()
TicketPurchasePublisher.publishTicketIssued() → publish(TicketIssuedEvent) → BusTicketPurchaseListener.onEvent → issuePremiumTicket(..., BusCity)

ACTUAL CODE SLICE

Subject는 listener 구현체를 모른다

```
// EventPublisher.java – Subject base
private final List<EventListener> listeners =
    new ArrayList<>();

public void subscribe(EventListener listener) {
    if (listener == null || listeners.contains(listener)) return;
    listeners.add(listener);
}

protected void publish(DomainEvent event) {
    for (EventListener listener : new ArrayList<>(listeners)) {
        listener.onEvent(event);
    }
}

// TicketPurchasePublisher.java – ConcreteSubject
public void publishTicketIssued(Reservation r,
                                Ticket ticket,
                                BusCity city) {
    publish(new TicketIssuedEvent(r, ticket, city));
}

// BusTicketPurchaseListener.java – ConcreteObserver
public void onEvent(DomainEvent event) {
    if (!(event instanceof TicketIssuedEvent e)) return;
    if (e.getRequestedBusCity() == null) return;
    busTicketingService.issuePremiumTicket(
        e.getReservation(),
        e.getTicket(),
        e.getRequestedBusCity());
}
```

감사합니다

Observer Pattern

부수효과는 호출자가 아니라 listener가 가져갑니다. 다음 iter4는 Singleton과 Factory Method로 전역 설정과 일정 종류별 생성 책임을 분리합니다.

- Questions?
- 4 Subjects
- 4 Listeners
- 4 Event types
- Air + Bus ticket
- Connecting + Multi-city
- Mileage payment

Singleton + Factory Method

- AppConfig Singleton — 폰트.언어.통화 전역 설정
- ItineraryFactory Factory Method — Direct/Connecting/MultiCity 생성
- 관리자 예외 환불 검토 경로 (Strategy + 관리자 승인)
- e-Ticket PDF 다운로드
- Reservation 상태 실시간 추적 (iter3 Observer 위에 구축)
- 마일리지 적립 (현재 차감만 구현됨)